Multiplayer Chat with Rooms

Real-time room-based communication architecture, components, and technology stack

What Was Built

- **Room-Based Live Chat:**
  Users join or create rooms by name with instant multi-client updates via Firestore real-time listeners.
- **Component-Driven Architecture:**
  Structured into 5 modular React components: RoomSelection, ChatRoom, MessageList, MessageItem, and MessageInput.
- **Timestamps & Visual Distinction:**
  Automatic message timestamps, sender-based bubble alignments, and hashed avatar colors.
- **Zero-Config Demo Mode:**
  Automatic BroadcastChannel fallback allowing reviewers to test multi-tab chat without API keys.

The Tech Stack

- **React 19 & Vite 6:**
  Functional components with hooks (useState, useEffect, useRef) with lightning-fast bundling (<1.2s build).
- **Firebase Firestore (v11):**
  Real-time subscriptions (onSnapshot), server timestamps, and decoupled service architecture.
- **Optimized Subcollection Schema:**
  rooms/{roomId}/messages eliminates Firestore composite index requirements out-of-the-box.
- **Modern Custom CSS:**
  Dark aesthetic, responsive mobile layout, smooth auto-scroll anchor, and zero heavy UI dependencies.

AI Reflection & Engineering Decisions

Where AI missed the mark, architectural corrections applied, and technical uncertainties

What the AI Got Wrong / What I Corrected

- **The Firestore Indexing Trap:**
  AI code defaulted to flat collection queries: where('room', '==', id) + orderBy('createdAt'). In Firestore, this crashes without a manual composite index. Corrected to subcollections (rooms/{id}/messages) for automatic indexing.
- **Missing Zero-Config Developer UX:**
  AI assumed live credentials; missing keys resulted in blank screens. Architected a seamless BroadcastChannel fallback for immediate multi-tab demoing.
- **State Management Over-Engineering:**
  AI often introduces Redux or Context for simple flows. Chose explicit props & local state.

Least Sure About & Future Iterations

- **Room Discovery at Scale:**
  Subcollections isolate room messages efficiently, but listing active rooms requires maintaining a separate root /rooms catalog or Collection Group Queries.
- **Presence & Typing Indicators:**
  Firestore document writes are too costly and slow for typing heartbeats. In production, Firebase Realtime Database is better suited for ephemeral presence.
- **Authentication & Impersonation:**
  Display names are currently unauthenticated. Production requires Firebase Auth integration with Firestore Security Rules to prevent spoofing.